```
        if (!idPosition.HasValue)                                    ❺
        {                                                            ❺
            return next;                                             ❺
        }                                                            ❺

        return async (invocationContext) =>                          ❻
        {
            var id = invocationContext                               ❼
                .GetArgument<string>(idPosition.Value);              ❼
            if (string.IsNullOrEmpty(id) || !id.StartsWith('f'))     ❼
            {                                                        ❼
                return Results.ValidationProblem(                    ❼
                    new Dictionary<string, string[]>                 ❼
                {{ "id", new[] { "Id must start with 'f'" }}});      ❼
            }                                                        ❼

            return await next(invocationContext);                    ❽
        };
    }
}
```

❶ The filter factory can handle endpoints with different method signatures.

❷ The context parameter provides details about the endpoint handler method.

❸ GetParameters() provides details about the parameters of the handler being called.

❹ Loops through the parameters to find the string id parameter and record its position

❺ If the id parameter isn't not found, doesn't add a filter, but returns the remainder of the pipeline

❻ If the id parameter exists, returns a filter function (the filter executed for the endpoint)

❼ If the id isn't valid, returns a Problem Details result

❽ If the id is valid, executes the next filter in the pipeline

The code in listing 5.11 is more complex than anything else we've seen so far, as it has an extra layer of abstraction. The endpoint middleware passes an `EndpointFilterFactoryContext` object to the factory function, which contains extra details about the endpoint in comparison to the context passed to a normal filter function. Specifically, it includes a `MethodInfo` property and an `EndpointMetadata` property.

NOTE You'll learn about endpoint metadata in chapter 6.